



课 程 作 业

数 据 挖 掘 与 机 器 学 习

程一娇

202321052006



PART 01

课 程 导 言

1、大数据的概念与4V特征

一、大数据定义

又称海量数据，指以不同形式存在于数据库、网络等媒介上，蕴含丰富信息的规模巨大的数据

二、4V特征

特征	含义	对应挑战
Volume（体量巨大）	数据量达TB/PB级别	大数据存储怎么做？
Variety（类型繁多）	日志、视频、图片、地理位置等	多源异构数据怎么有效融合？
Value（价值密度低）	大量数据中只有少量有价值信息	如何挖掘潜藏的有价值内容？
Velocity（处理速度快）	要求快速处理和分析	计算速度的要求

2、数据挖掘的基本概念

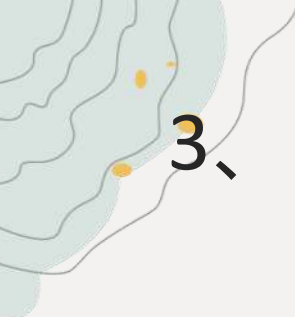
一、定义

数据挖掘是从大量的、不完全的、有噪声的、模糊的、随机的实际应用数据中，提取隐含的、事先未知的、潜在有用的信息和知识的过程。

二、数据 → 知识的过程

数据 → 信息 → 知识 → 情报 → 智慧

例：气温、湿度 → 天上一朵云 → 乌云会下雨 → 今天下午3:10下暴雨 → 及时收割或预防



3、数据挖掘的几个要点

- I. 大数据思维：利用数据的意识，开放、共享、合作
 - II. 大数据的收集与集成：理清与目标相关的数据，集中管理
 - III. 大数据的降维：通过抽样、变量筛选、PCA等方法减少数据量
 - IV. 大数据的分布式与并行处理：使用Map-Reduce、并行计算提高效率
- 

4、数据挖掘的六阶段模型

- I. 挖掘目标的定义：明确业务问题，设定可量化目标
- II. 数据的准备：选择数据、质量分析、预处理（清洗、集成、归约、变换）
- III. 数据的探索：描述统计、数据可视化、建模活动
- IV. 模型的建立：选择算法，训练模型，设置训练集/测试集
- V. 模型的评估：使用准确率、混淆矩阵、ROC曲线、回测等指标
- VI. 模型的部署：部署到业务系统，持续监控和优化

5、本章概念汇总

概念	简要说明
大数据 (Big Data)	规模巨大、类型多样的数据集合
数据挖掘 (Data Mining)	从数据中提取有用知识的过程
数据降维	减少变量数量, 如PCA、t-SNE
有监督学习	数据有标签, 如分类、回归
无监督学习	数据无标签, 如聚类、关联规则
结构化数据	表格形式, 如数据库
半结构化数据	如JSON、XML
非结构化数据	如文本、图像、视频
数据预处理	清洗、集成、归约、变换
数据标准化	将数据缩放到一定范围, 如0-1或Z-score
衍生变量	从原始变量组合生成的新变量

6、《白话机器学习》第1章读书笔记

一、内容摘要

- ① 数据准备：数据格式、变量类型（二值、分类、整型、连续）、变量选择、特征工程、缺失数据处理
- ② 算法选择：无监督学习（发现隐藏模式）、监督学习（做预测）、强化学习（不断改进）
- ③ 参数调优：避免过拟合（过于敏感）和欠拟合（过于迟钝），使用正则化
- ④ 模型评价：
 - 分类指标：准确率、混淆矩阵（TP, TN, FP, FN）
 - 回归指标：均方根误差（RMSE）
 - 验证方法：交叉验证（K折、留一法）

二、个人理解与收获

- ① 数据质量决定模型上限，数据预处理是关键第一步
- ② 不同任务（分类/回归/聚类）对应不同算法，不能混用
- ③ 过拟合是常见问题，正则化和交叉验证是有效解决手段
- ④ 混淆矩阵比单一准确率更能反映模型性能的细节



PART 02

回 归 算 法

1、数据回归方法的基本方法与步骤

基本方法

回归分析是处理变量之间**相关关系**的一种数学方法。
通过建立数学模型，描述因变量（Y）与自变量（X）之间的关系。

基本步骤

- 1.收集一组包含因变量和自变量的数据。
- 2.选定模型形式（如线性、非线性），利用数据按一定准则（如最小二乘法）估计模型系数。
- 3.使用统计分析方法（如 R^2 、F检验）评估模型效果。
- 4.判断模型是否适用于当前数据。
- 5.利用模型对因变量进行预测或解释。

2、一元与多元线性回归的数学模型

一元线性回归

模型形式：

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Y ：因变量， X ：自变量， β_0, β_1 ：回归系数， ϵ ：随机误差
目标：通过最小二乘法估计 β_0, β_1

多元线性回归

模型形式：

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon$$

$X_1, X_2, \dots, X_p$ ：多个自变量

可用矩阵形式表示：

$$Y = X\beta + \epsilon$$

评估指标： R^2 、F检验、残差分析等

2、Logistic回归的数学模型与特点

数学模型

用于因变量为**二分类** (0/1) 的情况

模型形式:

$$p(Y = 1 | X) = \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_k X_k}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_k X_k}}$$

对数变换后:

$$\ln\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \dots + \beta_k X_k$$

含义

预测某事件发生的概率 (0~1之间)

适用于二分类问题, 如: 是否贷款、股票涨跌

特点

1. 输出为概率, 可解释性强
2. 对数据分布要求较低
3. 适用于线性不可分问题

联系与区别

	线性回归	Logistic回归
区别	因变量为连续值	1、因变量为分类值 (0/1) 2、输出经过Sigmoid函数映射到[0,1]
联系	都基于回归思想, 使用自变量预测因变量	

3、《白话机器学习》第六章读书笔记

COO

内容摘要

回归分析用于预测连续值，如房价、股票价格

核心是找到最佳拟合线（趋势线）

常用方法：梯度下降法优化回归系数

回归系数表示各预测变量的影响力大小

相关系数（ r ）衡量变量间的线性关系强度（-1到1）

COO

个人理解

回归分析是机器学习中最基础、最直观的预测方法

通过历史数据学习变量间的关系，进而预测未来

梯度下降法是一种通用的优化方法，适用于多种模型

标准化处理（如Z-score）可以消除量纲影响，提升模型可比性

COO

局限性

对异常值敏感

多重共线性会影响系数估计

不能解释因果关系，只能说明相关性



PART 03

分 类 算 法

1、分类算法的基本逻辑与基本概念

基本概念

训练集：用来训练模型的原始数据（划分出一部分）；

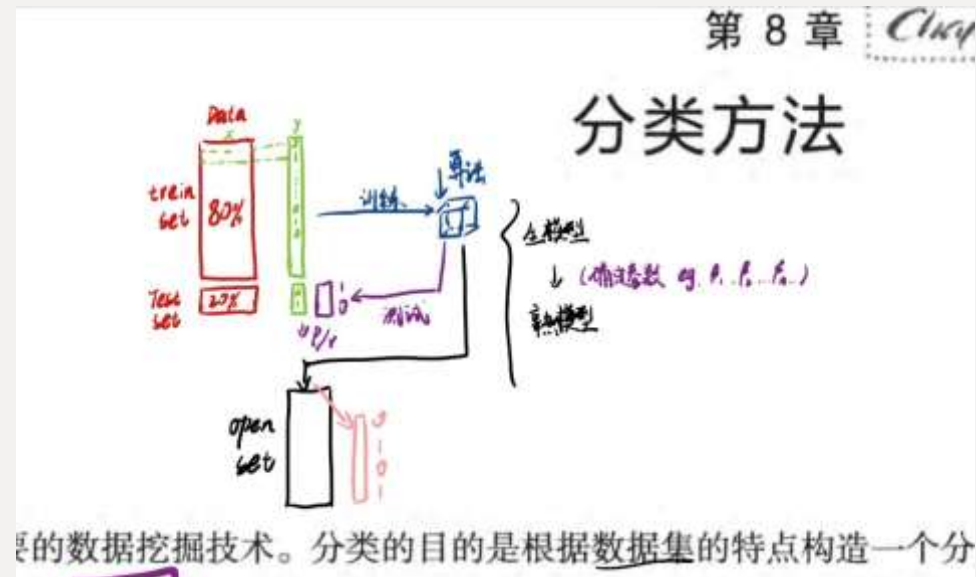
类标签：原始数据中已经给每个记录（对象）标记的类别；

测试集：用来测试模型的原始数据（划分出的一部分）；

开放集：使用训练好的模型（分类器）进行分类工作的新数据；

基本逻辑

对现有的数据进行学习，得到一个目标函数或模型（分类器），该模型能把未知类别的样本映射到给定的类别中。



2、KNN算法的思想与算法步骤

算法思想

K-近邻 (K-Nearest Neighbor, KNN) 算法是一种基于实例的分类方法 (①K ②数据空间的分布特征)

算法步骤

01

通过计算待分类样品到每个训练样例到的距离 (度量)

02

取和待分类样品距离最近的K个训练样例 (邻居)

03

K个样品中哪个类别的训练样例占多数 (统计)

04

则待分类样品就属于哪个类别 (投票)

3、《白话机器学习》第七章KNN读书笔记

COO

基本原理:

算法根据周围最近的 K 个数据点的类别，对目标点进行分类。 K 表示参考的邻居数量，是唯一关键参数。分类规则：少数服从多数， K 个邻居中哪类最多，目标点就属于哪类。

COO

K 值的影响:

K 太小: 模型过度敏感，噪声影响大，容易过拟合。

K 太大: 模型过于粗糙，忽略局部模式，容易欠拟合。

二分类推荐使用奇数，避免投票出现平局。

COO

局限性:

①类别不平衡: 小类别容易被大类别覆盖→加权投票法（确保较近数据点类别权重比较远的更大）

②预测量变多: 特征维度太高时效果明显下降→降维技巧

3、《白话机器学习》第九章决策树读书笔记

COO

决策树结构：

根节点：第一层、最重要的判断条件。

内部节点：每一层的判断条件。

分支：判断后的走向。

叶节点：最终分类结果。

COO

生成方法：

①确定一个二元选择题，它能够把数据点拆分为两组，并最大限度的提高每组数据点的同质性。

②对每个叶节点重复步骤1，直到满足停止条件。

③终止条件有**多个**：交叉验证法

COO

局限性：

不稳定：数据微小变动就可能生成完全不同的树。容易过拟合，在新数据上泛化能力差。

不确定：单次最优选择不一定带来全局最优结果。

→随机森林、梯度提升



PART 04

聚 类 算 法

1、簇的概念

对聚类理解的三个层次

层次	描述
层次一	“物以类聚，人以群分”——感官认知
层次二	同一个簇中的对象 相似度较高 ，与其他簇中的对象相似度较低——逻辑判断
层次三	同一个分组中的记录 越近越好 ，而不是分组中的记录越远越好

核心思想

聚类的思维是：感观 → 逻辑 → 量化，做研究、做事情也是如此。

簇的形象特征

1. 簇在空间分布上不是确定的，呈各种不同的形状
2. 在二维平面里可以有各种不同的形状（圆形、椭圆形、条带状等）
3. 在多维空间里形状更加多样

2、类的度量方法

01

欧式距离)

二维平面上点 $a(x_1, y_1)$ 与 $b(x_2, y_2)$ 间的欧氏距离:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

特点: 衡量两点之间的**直线距离**, 值越小越相似。

02

余弦相似度

二维空间中向量 $A(x_1, y_1)$ 与向量 $B(x_2, y_2)$ 的夹角余弦公式:

$$\cos \theta = \frac{x_1 x_2 + y_1 y_2}{\sqrt{x_1^2 + y_1^2} \cdot \sqrt{x_2^2 + y_2^2}}$$

特点: 衡量两个向量的**方向一致性**, 值越接近1越相似 (与长度无关)。

03

两者对比

度量方法	适用场景	关注点
欧式距离	数值特征相近性	绝对距离
余弦相似度	文本、偏好相似性	方向一致性

3、K-means算法的原理与步骤

01

工作原理

- ①随机从数据集中选取 k 个点，每个点初始代表每个簇的聚类中心
- ②计算剩余各个样本到聚类中心的距离，将它赋给最近的簇
- ③重新计算每一簇的平均值，作为新的聚类中心
- ④重复②-③，直到相邻两次调整没有明显变化（点的归属或聚类中心不再改变）

02

算法特点

特点	说明
K值难确定	K需事先给定，难以估计
对初始值敏感	不同初始中心可能导致不同结果（不稳定）
计算量大	数据量大时时间开销大，但仍是大数据集的优先选择（简洁有效）
无法处理交叉类别	一个对象只能属于一个簇，不能属于多个类
对离散点敏感	异常值可能影响聚类结果

4、层次聚类（HC）算法过程

01

两种基本思想

- ① 凝聚法（自底向上）：开始时每个点自成一类，然后不断合并距离最近的两类，直到达到指定的类别数 k 。
- ② 分裂法（自顶向下）：开始时所有点为一类，然后不断分裂，直到达到指定的类别数 k 。

03

簇间距离的计算方法

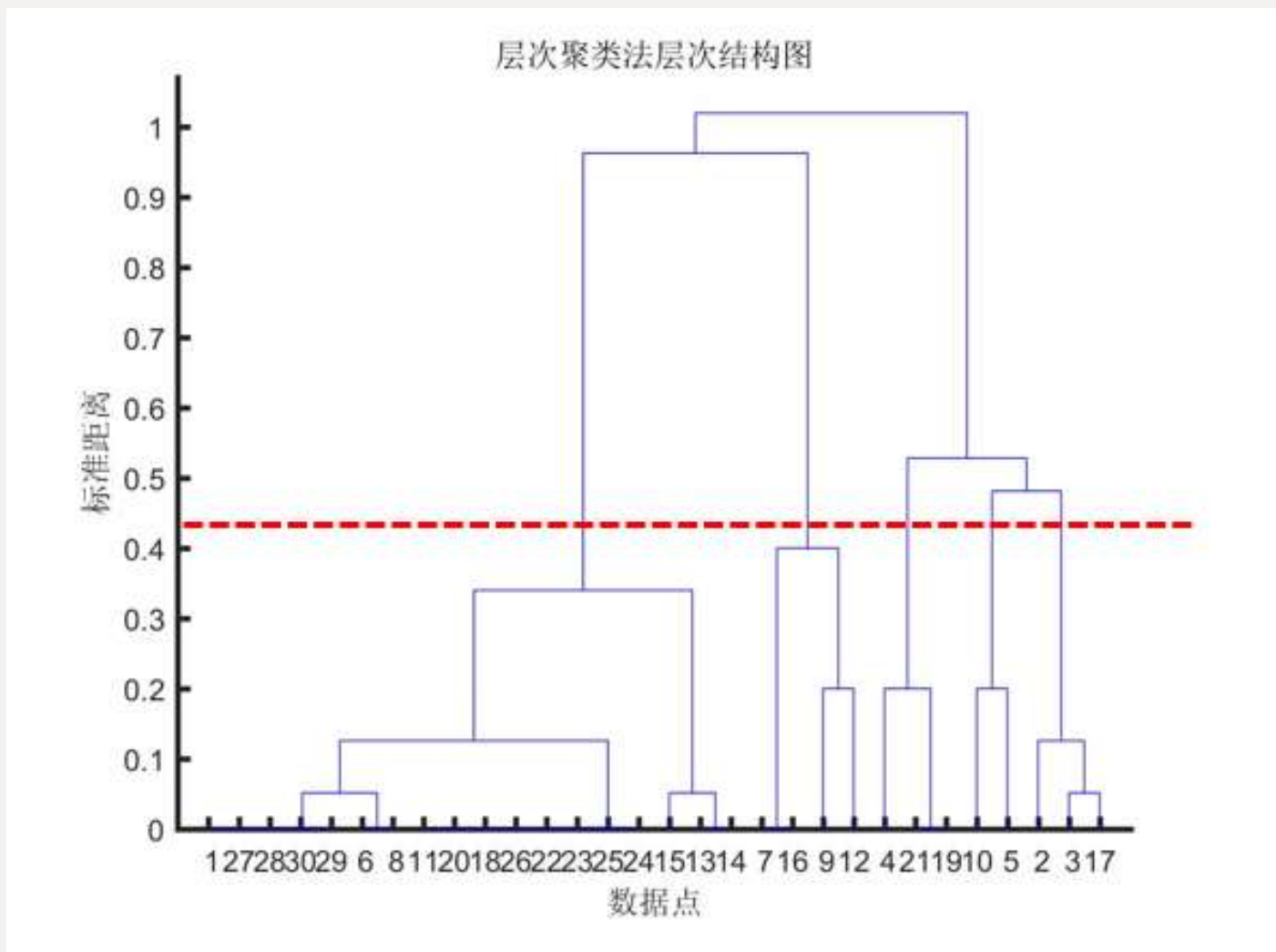
- ① 单链法：两个簇中最近的两个点之间的距离。
- ② 全链法：两个簇中最远的两个点之间的距离。
- ③ 平均链法：两个簇中所有点对之间的平均距离。

02

凝聚法的具体步骤

- ① 将数据集中的每一个样本点视为一个独立的簇。
- ② 找到分属不同簇且距离最近的两个点（或两个簇）。
- ③ 将这两个簇合并成一个新的簇。
- ④ 重复第2、3步，直到簇的数量减少到预设的 k 值为止。

5、HC聚成5类结果图



6、K-means与HC算法对比总结

两种思想

对比维度	K-means	层次聚类 (HC)
聚类方式	划分式	层次式 (凝聚/分裂)
是否需预先指定K	是 (K值难确定)	否 (可从树状图确定)
计算复杂度	较高	很高
适用数据规模	大数据集 (简洁有效)	小数据集
结果稳定性	不稳定 (对初始值敏感)	稳定
可视化	平面/三维散点图	树状图 (dendrogram)
优势	快速、简洁	层次结构清晰、可解释性强
劣势	无法处理交叉类别、对离散点敏感	计算量大、不适合大数据

选择建议

数据量大、需要快速聚类 → K-means

数据量小、需要清晰层次结构 → HC

不确定K值、希望探索数据结构 → HC

7、《白话机器学习》第2章读书笔记

01

本章核心内容

- A. 找出顾客群：通过识别共同的喜好或特征，把顾客分成不同群体，用于精准营销和个性化推荐。
- B. 示例：影迷的性格特征：使用外向型、尽责型、情绪型、开放型四个维度对影迷进行聚类，发现不同性格的人喜欢不同类型的电影。
- C. 如何定义群组：需要解决两个问题——有多少个群组（用陡坡图判断）、每个群组中有谁（通过迭代分配确定）。
- D. 陡坡图法：横轴是群组数量 K ，纵轴是群组内散度。曲线拐弯处的 K 值就是最佳群组数量。
- E. K-means迭代的四个步骤：
 - ① 猜测 K 个群组的中心点。
 - ② 将每个数据点分配给最近的中心点。
 - ③ 根据分配结果重新计算每个群组的中心点。
 - ④ 重复第2、3步，直到群组成员不再变化。

7、《白话机器学习》第2章读书笔记

02

K-means的局限性

- ① 每个数据点只能属于一个群组，无法处理处于两个群组边界的情况。
- ② 假定群组是正圆形的，不适合椭圆形或不规则形状的群组。
- ③ 假定群组是离散的，不允许群组重叠或嵌套。

03

个人理解与收获

- ① K-means是最简单、最直观的聚类算法，适合快速了解数据的整体结构。
- ② 陡坡图是一个实用的工具，可以帮助我们在不确定K值的情况下做出合理选择。
- ③ 使用K-means时要心中有数：它只能处理圆形、离散、不重叠的群组。
- ④ 一个好的策略是：先用K-means快速探索数据，发现问题，再用更高级的聚类方法（如高斯混合模型、密度聚类）深入分析。
- ⑤ K-means在实际中应用非常广泛，客户分群、商品分类、图像压缩等场景都能看到它的身影。



PART 05

关联规则

1、关联选股的背景知识

01

基本思想

- ① 从众多个股中选择具有增长潜力的个股难度较大，但行业数量较少，选对行业的可能性更高。
- ② 股市中常常出现同行业股票普涨或普跌的现象。
- ③ 只要选对行业，无论怎么选个股，都可能盈利。

03

为什么用关联规则？

- ① 关联规则可以挖掘不同行业之间涨跌的内在联系。
- ② 通过历史数据发现行业联动的规律，指导未来的投资决策。

02

核心逻辑

- ① 寻找具有联动关联的行业。
- ② 当某个行业出现涨势后，其关联行业还未开始涨，则从其关联行业中选择典型个股买入。
- ③ 这是一种提前埋伏的投资策略。

04

应用价值

- ① 降低选股难度（从几千只股票缩小到几十个行业）。
- ② 提高投资效率（把握行业轮动机会）。
- ③ 可与其他选股策略结合使用。

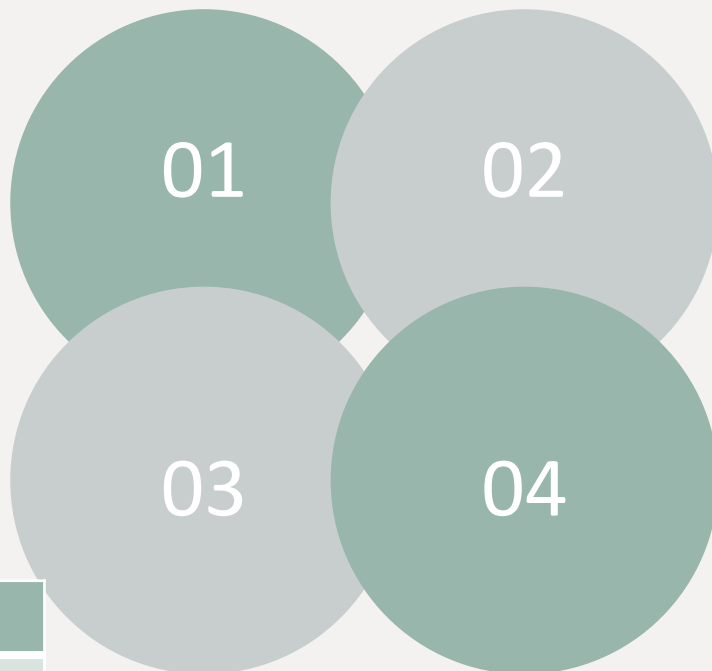
2、数据说明——矩阵的行与列含义

数据来源

通过交易系统或公共股票数据，获取各行业在多个交易日内的涨跌情况。

矩阵的行与列含义

维度	含义	说明
行	一个交易日（或一个时间窗口）	每一行代表一个事务（Transaction）
列	一个行业	每一列代表一个项（Item）
矩阵元素	该行业在该交易日是否表现良好	1表示好（涨势好），0表示不好



数据预处理

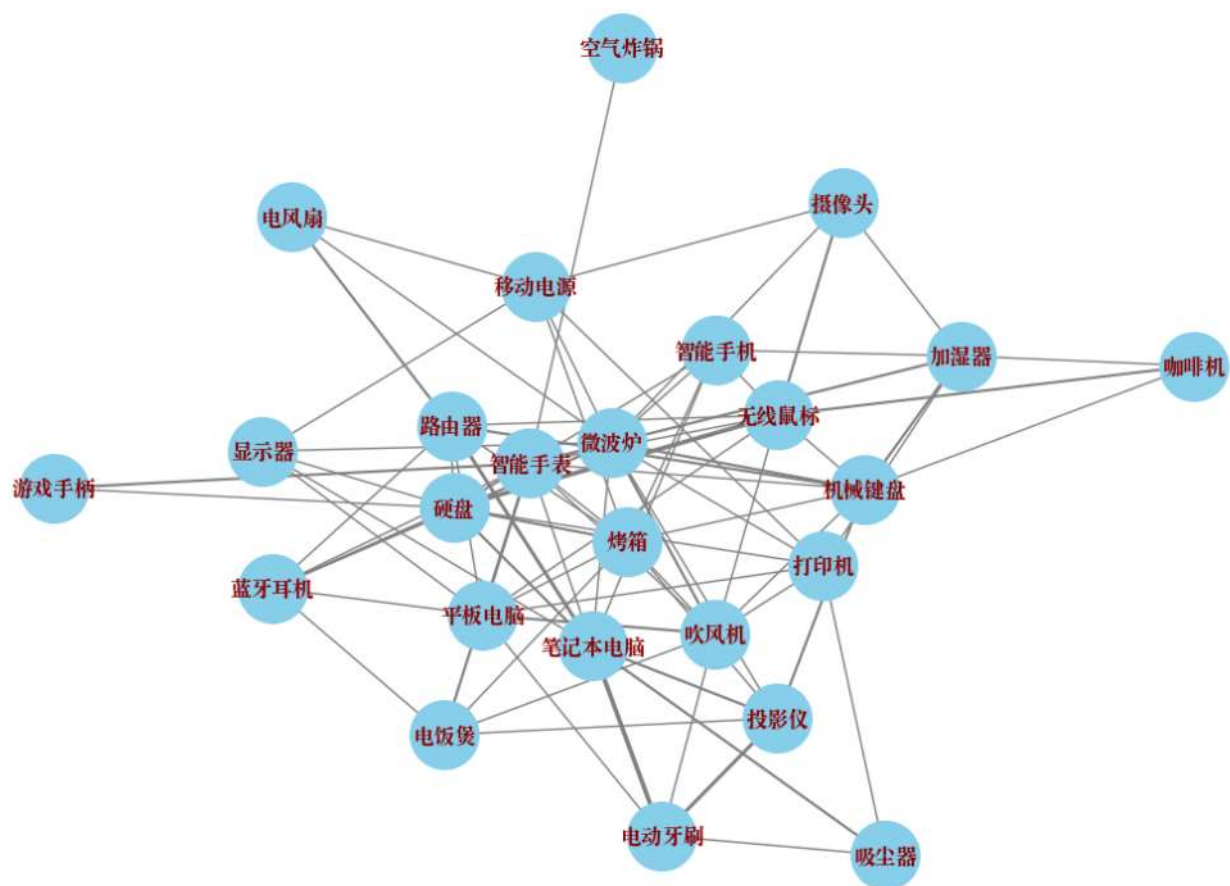
- ① 定义一个标准来界定哪些行业算“涨势好的行业”。
- ② 例如：10个交易日内，行业涨幅超过大盘涨幅5%的行业，标记为“好行业”（1），否则标记为“0”。

关键理解

- ① 这就是一个0-1矩阵，是关联规则挖掘的标准输入格式。
- ② 每一行是一个“购物篮”，只不过里面的“商品”换成了“行业”。
- ③ 关联规则挖掘的目标：找出哪些行业经常同时表现良好。

3、程序运行

序号	商品1	商品2	商品3
1	摄像头	智能手机	智能手表
2	路由器	智能手机	移动电源
3	平板电脑	电风扇	空气炸锅
4	烤箱	吹风机	空气炸锅
5	笔记本电脑	智能手机	打印机
6	显示器	智能手表	机械键盘
7	游戏手柄	显示器	无线鼠标
8	蓝牙耳机	吸尘器	无线鼠标
9	无线鼠标	加湿器	电风扇
10	吹风机	路由器	电饭煲



4、补充思考与拓展

01

优点

- ① 思路清晰，易于理解和实施。
- ② 降低选股难度，从行业层面把握机会。
- ③ 可以与其他技术分析、基本面分析方法结合。

03

拓展方向

- ① 使用FP-Growth算法处理更大规模的数据。
- ① 引入更多维度：资金流向、政策利好、行业景气度等。
- ② 构建行业关联网络，进行可视化分析。
- ③ 结合时间序列分析，捕捉行业轮动的时序特征。

02

缺点与注意事项

- ① 关联规则挖掘的是相关性，不是因果关系。
- ② 行业联动关系可能随时间变化，需要定期更新模型。
- ③ 历史规律不一定代表未来，需要结合市场环境判断。
- ④ 最小支持度和置信度的设定需要经验和调试。



PART 06

一元回归

2、显示生成“熟模型”

```
[55]: 0.9529481969048356
```

```
[56]: # 使用划分训练集、测试集的方法来处理线性回归问题
```

```
[57]: regressor = regressor.fit(X_train, Y_train)
      b1 = regressor.coef_
      b0 = regressor.intercept_
      print(f"熟模型方程: y = {b0:.4f} + {b1[0]:.4f} * x") # .fit # 训练之后, 就成了熟模型 # 问“X_train 是
```

熟模型方程: $y = 2.0182 + 9.9107 * x$

```
[28]: regressor
```

```
[28]: LinearRegression()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the

如何显示“熟模型”？ b0和b1怎么获得？

“熟模型”指的是训练完成后得到的线性回归方程。通过训练好的 regressor 对象直接获取截距b0和系数（斜率） b1

```
b1 = regressor.coef_
```

```
b0 = regressor.intercept_
```

3、评价结果好坏

- 2 b0 和 b1怎么生成

```
[72]: b1 = regressor.coef_ # 系数  
b1
```

```
[72]: array([9.91065648])
```

```
[73]: b0 = regressor.intercept_ #截距 y = 2 + 9.9x  
b0
```

```
[73]: np.float64(2.0181600414346974)
```

- 3 评价结果的好坏 R^2

```
[74]: R2_train = regressor.score(X_train,Y_train)  
print(f"训练集  $R^2$  = {R2_train:.4f}")
```

```
训练集  $R^2$  = 0.9516
```

```
[75]: R2_test = regressor.score(X_test,Y_test)  
print(f"测试集  $R^2$  = {R2_test:.4f}")
```

```
测试集  $R^2$  = 0.9455
```

如何评价结果的好坏？ R^2 是什么？

使用决定系数 R^2 评价结果的好坏。 R^2 取值范围0~1，越接近1

表示模型拟合越好。训练集 $R^2=0.9516$ ，测试集 $R^2=0.9455$ ，两

者都很高且接近，说明模型泛化能力优秀。

4、调整训练/测试比例

```
[76]: from sklearn.model_selection import train_test_split
```

```
[77]: test_sizes = [0.1, 0.2, 0.3, 0.4]
print("测试集比例\t训练集R²\t测试集R²")
print("-" * 40)
for ts in test_sizes:
    X_tr, X_te, Y_tr, Y_te = train_test_split(X, Y, test_size=ts, random_state=42)
    model_temp = LinearRegression()
    model_temp.fit(X_tr, Y_tr)
    r2_train = model_temp.score(X_tr, Y_tr)
    r2_test = model_temp.score(X_te, Y_te)
    print(f"{ts}\t\t\t{r2_train:.4f}\t\t\t{r2_test:.4f}")
```

测试集比例	训练集R²	测试集R²
0.1	0.9488	0.9740
0.2	0.9491	0.9678
0.3	0.9536	0.9488
0.4	0.9491	0.9512

```
[78]: _train, Y_test = train_test_split(X, Y, test_size = 0.2, random_state = 0) # random_state = 0 每次运行分
```

调整训练/测试比例对R²的影响

通过循环 test_size = [0.1, 0.2, 0.3, 0.4] 观察R²变化。

实验显示：测试集R²分别为0.974、0.968、0.949、0.951，整体稳定在0.95附近。当测试集过小时评估结果可能偏乐观；过大时训练数据减少导致R²轻微下降。一般取0.2或0.25较为平衡。



PART 07

多元回归

1、数据集的特征

01

属性

数据集 50_Startups.csv 包含 5个属性（列）

	R&D Spend-X1	Administration-X2	Marketing Spend-X3	State-X4	Profit-Y
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

03

可视化State和Profit之间的关系

- A. 上图展示了三个投入变量与利润之间的关系，并按不同州着色：
- B. 研发投入与利润呈明显的正相关关系，且相关性最强；
- C. 行政管理费用与利润关系不明显，回归线接近水平；
- D. 市场营销投入与利润也呈正相关，但弱于研发投入；
- E. 不同州的点在图中混合分布，说明州变量有一定影响，但并非决定性因素。

02

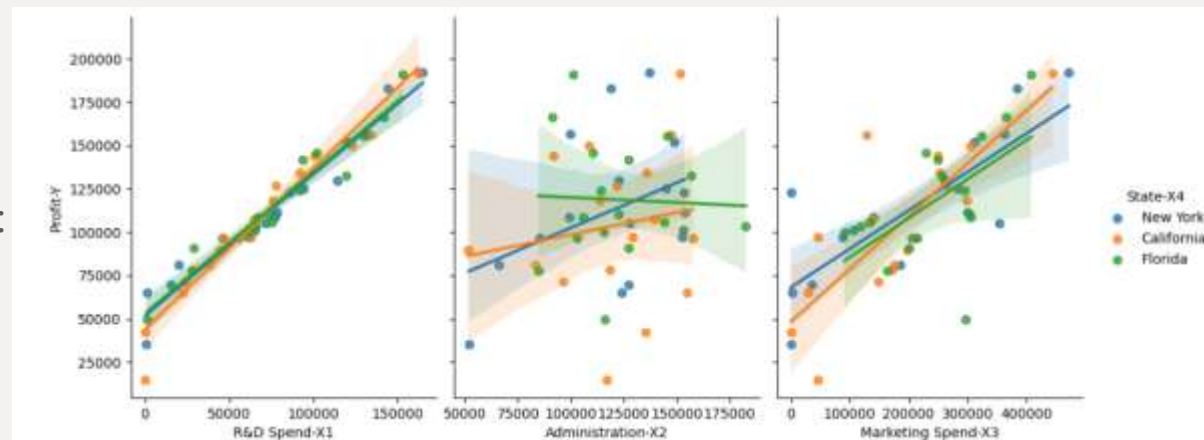
分类变量

分类变量是：State（州）

该属性有三个取值：

- 1、New York（纽约）
- 2、California（加利福尼亚）
- 3、Florida（佛罗里达）

分类变量本身不是数值，不能直接用于回归模型，需要进行编码转换（如One-Hot编码）。



2、分类变量转化为onehot

多元线性回归要求所有自变量都是**数值型**。

State 列是文本（New York / California / Florida），不能直接放入模型，需要转换为数值形式。

One-Hot编码 是最常用的方法：将一个分类变量扩展为多个二值变量（0或1）。

转化过程

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.compose import ColumnTransformer
labelencoder_X = LabelEncoder() # 先用LabelEncoder把分类变量标签化，就是用数字标号代替文字
X[:, 3] = labelencoder_X.fit_transform(X[:, 3])
```

第一步：导入所需库，用 LabelEncoder 将文本标签转换为数字标签

```
ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [3])], remainder='passthrough')
X = np.array(ct.fit_transform(X))
***由于现在的onhotencoder会默认对整个数据集进行处理，并且没有了categorical_features，所以利用Co
***其实也可以通过pandas中的get_dummies()来进行onehot编码***
```

```
X = X[:, 3:6]
```

第二步：用 ColumnTransformer + OneHotEncoder 将数字标签转换为One-Hot向量，删除第一列One-Hot编码，避免多重共线性

转化总结

先用 LabelEncoder 把文字转成数字，再用 OneHotEncoder 把数字转成多列二值变量，最后删除一列避免虚拟变量陷阱。

3、评估结果 R^2

在多元线性回归中，常用的评估指标是 R^2 （决定系数，Coefficient of Determination）。

R^2 的含义

R^2 表示：模型能够解释因变量（Profit）中多大比例的变化。

- ① 取值范围：0 到 1
- ② $R^2 = 1$ ：模型完美拟合，预测值完全等于真实值
- ③ $R^2 = 0$ ：模型没有任何解释力，相当于直接用平均值预测
- ④ R^2 越高，模型拟合效果越好

```
R2 = regressor.score(X,Y)
```

```
R2
```

```
0.9490884301964866
```

约 94.9% 的利润变化可以被这个回归模型解释

只有约 5.1% 是模型无法解释的误差

$R^2 = 0.949$ ，说明模型能够解释约94.9%的利润变化，拟合效果很好。

4、如何获得系数（权重）和截距？

01

如何获得权重和截距？

数据集 50_Startups.csv 包含 5个属性（列）

```
bi = regressor.coef_ # 系数（权重）
bi

array([0.77884104, 0.0293919 , 0.03471025])

b0 = regressor.intercept_
b0

np.float64(42989.00816508669)
```

03

截距的含义

b0 = 42989.01

含义：当所有自变量（R&D、Administration、Marketing）都为0时，利润的基准值为约42989。

在实际业务中，“所有投入都为0”的情况不会发生，截距更多是一个数学上的参考值。

02

每个权重的含义

变量	系数值	含义解读
R&D Spend	0.7788	研发投入每增加1单位，利润增加约0.78单位
Administration	0.0294	行政管理费用每增加1单位，利润增加约0.03单位（影响很小）
Marketing Spend	0.0347	市场营销投入每增加1单位，利润增加约0.03单位

最终回归方程为： $\text{Profit} = 42989 + 0.78 \times \text{R\&D} + 0.03 \times \text{Administration} + 0.03 \times \text{Marketing}$ ，其中研发投入对利润影响最大。

5、分类变量

多元线性回归的输入要求所有自变量都是数值型。

分类变量（如State）是文本或类别标签，不能直接放入回归模型中计算。

01

分类变量带来的主要影响

影响	说明
1. 无法直接计算	回归模型中的数学运算（加减乘除）无法处理“New York”这样的文本
2. 需要编码转换	必须将分类变量转换为数值形式（如One-Hot编码）才能放入模型
3. 增加特征维度	一个k个类别的分类变量，会扩展为k-1个新变量（避免虚拟变量陷阱）
4. 改变模型解释方式	编码后的系数不再表示“单位变化的影响”，而是表示“相对于基准类别的差异”
5. 可能引入多重共线性	如果不删除一列One-Hot编码，会导致变量之间完全相关，模型无法求解

```
x = x[:,3:6]
```

第二步：用 ColumnTransformer + OneHotEncoder 将数字标签转换为One-Hot向量，删除第一列One-Hot编码，避免多重共线性

5、分类变量

02

本次案例的具体影响

以 State 为例（3个类别：New York、California、Florida）：

处理后：

- ① 原来的1列 State 被移除
- ② 增加 State_California、State_Florida 两列（二值变量，0或1）
- ③ New York 成为基准类别（当两列都为0时，表示New York）

回归方程实际变成：

$$\text{Profit} = b_0 + b_1 \times \text{R\&D} + b_2 \times \text{Admin} + b_3 \times \text{Marketing} + b_4 \times \text{State_CA} + b_5 \times \text{State_FL}$$

系数解释：

b4（State_CA的系数）：表示California相对于New York的利润差异

b5（State_FL的系数）：表示Florida相对于New York的利润差异

转化总结

分类变量不能直接用于多元线性回归，必须通过One-Hot编码转换为数值型变量，并删除一列避免多重共线性；否则要么程序报错，要么模型产生错误的数学含义。

5、分类变量

01

直接Label化后计算

直接用这组0、1、2进行回归（不做One-Hot），程序可以运行，不会报错，但是不合理

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.compose import ColumnTransformer
labelencoder_X = LabelEncoder() # 先用LabelEncoder把分类变量标签化，就是用数字标号代替文字
X[:, 3] = labelencoder_X.fit_transform(X[:, 3]) # 只对 State 列做编码

X[0:10]
```

```
array([[165349.2, 136897.8, 471784.1, 2],
       [162597.7, 151377.59, 443898.53, 0],
       [153441.51, 101145.55, 407934.54, 1],
       [144372.41, 118671.85, 383199.62, 2],
       [142107.34, 91391.77, 366168.42, 1],
```

不能直接对分类变量做LabelEncoder（变成0、1、2）后放入回归模型是不合理的，因为这会错误地给类别强加数值大小和倍数关系，导致回归系数含义失真。

正确的做法是使用One-Hot编码。LabelEncoder只能作为One-Hot前的中间步骤，不能替代One-Hot直接用于回归。

02

不合理的原因

① 强加数值大小关系

模型会自动认为 $2 > 1 > 0$ ，即 New York > Florida > California。但实际上这三个州只是类别不同，不存在谁“大于”谁的关系。

① 错误的倍数关系

回归方程中会出现 $b \times \text{State}$ 数字的项。

例如：

State=2 的影响 = $b \times 2$

State=1 的影响 = $b \times 1$

模型会错误地认为 New York 的影响是 Florida 的 2 倍，这毫无道理。

① 编码结果不稳定

改变数据读取顺序，数字分配会改变，模型结果也会随之变化。结论不可信。

PART 08

K N N 算 法

1、性别因素Acc值的变化

未加入性别

```
# 我们获取数据的属性列，就是2列，年龄列，3列收入列
X = dataset.iloc[:, [2, 3]].values
```

```
X[0:5]
```

```
array([[ 19, 19000],
       [ 35, 20000],
       [ 26, 43000],
       [ 27, 57000],
       [ 19, 76000]])
```

accuracy

0.95

对比项	修改前	修改后
使用的特征	Age, Salary	Age, Salary, Gender_Male
准确率	0.95	0.95 (不变)

加入性别

```
dataset = pd.read_csv('Social_Network_Ads.csv') # 我们使用pandas读取表
```

```
dataset = pd.get_dummies(dataset, columns=['Gender'], drop_first=True)
X = dataset.iloc[:, [1, 2, 4]].values
y = dataset['Purchased'].values
```

```
dataset.head() # 我们看一下这个数据长什么样子
```

```
# 我们获取数据的属性列，就是2列，年龄列，3列收入列
#X = dataset.iloc[:, [2, 3]].values
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

accuracy

np.float64(0.95)

对性别进行 One-Hot 编码后，KNN 模型的准确率仍为 0.95，与原模型（仅使用年龄和收入）一致，说明性别不是影响购买决策的关键因素。

2、比较KNN与Logistic的结果差异

原代码 (KNN)

```
from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p = 2)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
```

在本数据集中，KNN 的准确率为 95%，Logistic 回归的准确率为 91.25%。KNN 表现优于 Logistic，说明数据分布可能不是完全线性可分的，KNN 作为非线性模型更具优势。

Logistic回归代码

```
# ===== KNN 分类器 =====
from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
classifier.fit(X_train, y_train)
# ===== 预测 =====
y_pred = classifier.predict(X_test)
# ===== 混淆矩阵和准确率 =====
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
accuracy = (cm[0,0] + cm[1,1]) / (cm[0,0] + cm[0,1] + cm[1,0] + cm[1,1])
print("KNN 准确率:", accuracy)
# ===== 在这里添加 Logistic 回归代码 =====
from sklearn.linear_model import LogisticRegression
logistic = LogisticRegression(random_state=0)
logistic.fit(X_train, y_train)
y_pred_logistic = logistic.predict(X_test)
cm_logistic = confusion_matrix(y_test, y_pred_logistic)
accuracy_logistic = (cm_logistic[0,0] + cm_logistic[1,1]) / (cm_logistic[0,0] +
print("Logistic 准确率:", accuracy_logistic)
# =====
```

KNN 准确率: 0.95

Logistic 准确率: 0.9125

3、训练集比例如何影响结果

实验代码

```
: #X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=0)
#X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
#X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=0)
```

accuracy

np.float64(0.89375)

实验结果

测试集比例	训练集大小	测试集大小	准确率 (KNN)
10%	360	40	0.925
20%	320	80	0.95
30%	280	120	0.917
40%	240	160	0.894

实验比较了测试集比例分别为 10%、20%、30%、40% 时 KNN 模型的准确率。结果显示，测试集比例为 20% 时准确率最高（95%）。比例过小（10%）评估不稳定，比例过大（30%-40%）导致训练不足、准确率下降。因此，训练集与测试集的最佳比例约为 8:2。

4、超参数K如何影响结果

实验代码

```
# K=1
classifier = KNeighborsClassifier(n_neighbors=1, metric='minkowski', p=2)
# K=3
classifier = KNeighborsClassifier(n_neighbors=3, metric='minkowski', p=2)
# K=5
classifier = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
# K=7
classifier = KNeighborsClassifier(n_neighbors=7, metric='minkowski', p=2)
# K=9
classifier = KNeighborsClassifier(n_neighbors=9, metric='minkowski', p=2)
```

实验结果

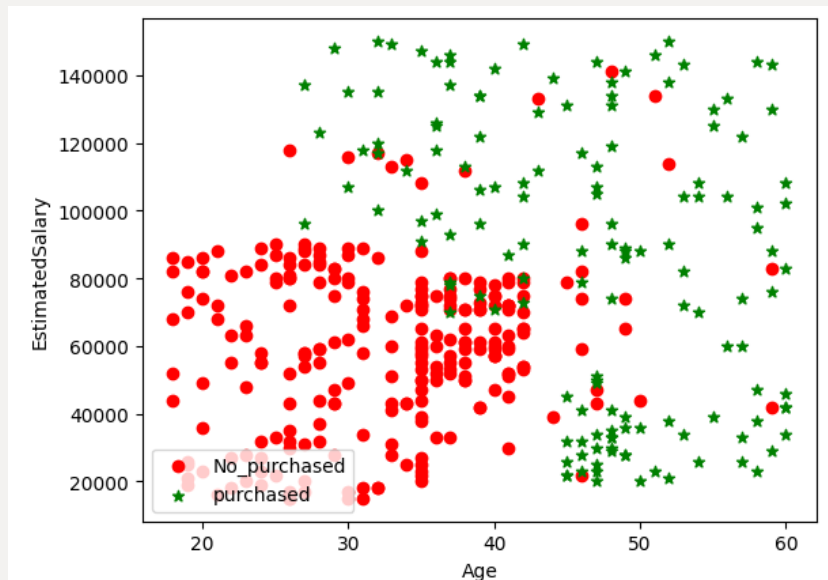
K值	准确率 (test_size=0.4)	说明	准确率 (test_size=0.2)
1	0.88125	容易过拟合，对噪声敏感	0.8875
3	0.90000	有所改善	0.9500
5	0.89375	略有波动	0.9500
7	0.90000	表现良好	0.9500
9	0.9125	最佳	0.9500

实验比较了 K=1、3、5、7、9 时 KNN 模型的准确率（测试集比例 40%）。结果显示：K=1 时准确率最低（88.1%），出现过拟合；K=9 时准确率最高（91.25%）。因此，在本数据集上，K=9 是最佳选择。同时发现，测试集比例越大，K 值对模型性能的影响越明显。

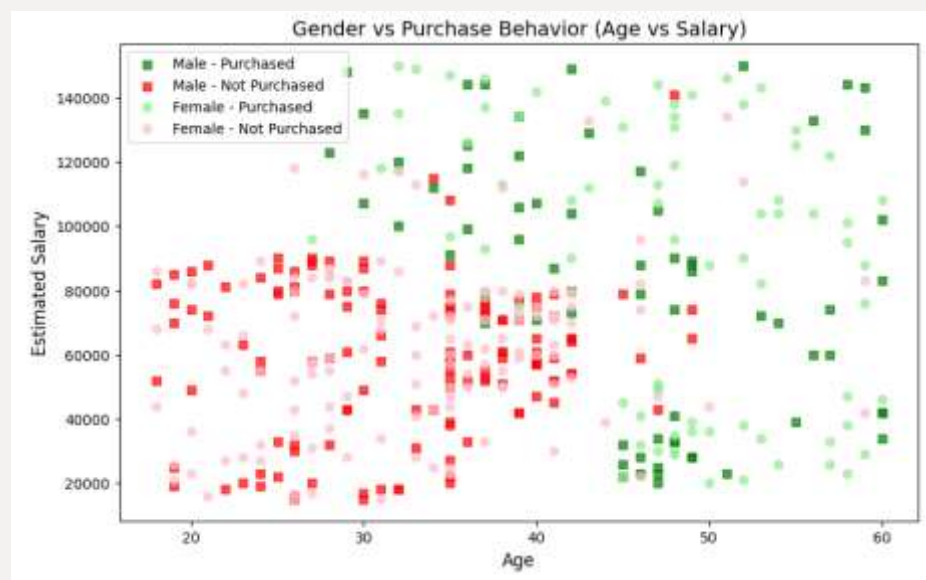
5、将性别变量加入可视化

原代码中的可视化

```
plt.scatter(x0[:,2], x0[:,3], c = 'red', marker = 'o', label = 'No_purchased')
plt.scatter(x1[:,2], x1[:,3], c = 'green', marker = '*', label = 'purchased')#
plt.xlabel('Age')
plt.ylabel('EstimatedSalary')
plt.legend(loc=3)
plt.show()
```



```
# 男性：用方形，购买用绿色，不购买用红色
plt.scatter(male_purchased['Age'], male_purchased['EstimatedSalary'],
            c='green', marker='s', label='Male - Purchased', alpha=0.7)
plt.scatter(male_not_purchased['Age'], male_not_purchased['EstimatedSalary'],
            c='red', marker='s', label='Male - Not Purchased', alpha=0.7)
# 女性：用圆形，购买用浅绿色，不购买用粉色
plt.scatter(female_purchased['Age'], female_purchased['EstimatedSalary'],
            c='lightgreen', marker='o', label='Female - Purchased', alpha=0.7)
plt.scatter(female_not_purchased['Age'], female_not_purchased['EstimatedSalary'],
            c='pink', marker='o', label='Female - Not Purchased', alpha=0.7)
plt.xlabel('Age', fontsize=12)
plt.ylabel('Estimated Salary', fontsize=12)
plt.title('Gender vs Purchase Behavior (Age vs Salary)', fontsize=14)
plt.legend(loc='best')
plt.show()
```





PART 09

决策树

1、性别特征的影响

```
X = dataset.iloc[:, [2, 3]].values  
y = dataset.iloc[:, 4].values
```

```
array([[53, 5],  
       [ 2, 20]])
```

```
[14]:
```

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])  
accuracy
```

```
[14]:
```

```
0.9125
```



```
dataset['Gender'] = dataset['Gender'].map({'Male': 0, 'Female': 1})  
X = dataset.iloc[:, [1, 2, 3]].values  
y = dataset.iloc[:, 4].values
```

```
array([[54, 4],  
       [ 2, 20]])
```

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])  
accuracy
```

```
np.float64(0.925)
```

	加入前	加入后
准确率	0.9125	0.925
提升	—	+1.25%

性别对购买行为具有一定的区分度，加入后模型能更准确地捕捉用户特征，准确率小幅提升。

2、特征缩放对随机森林的影响

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
print("有特征缩放: ")
cm
```

有特征缩放:

```
array([[55,  3],
       [ 2, 20]])
```

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
print("准确率: ", accuracy)
```

准确率: 0.9375



```
#from sklearn.preprocessing import StandardScaler
#sc = StandardScaler()
#X_train = sc.fit_transform(X_train)
#X_test = sc.transform(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
print("无特征缩放: ")
print(cm)
```

无特征缩放:

```
[[54  4]
 [ 2 20]]
```

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
print("准确率: ", accuracy)
```

准确率: 0.925

实验结果显示，删除特征缩放后，随机森林模型的准确率从93.75%下降到92.5%，降低了1.25个百分点

Age (19~35) 与Salary (19000~150000) 量纲差异大，不缩放时Salary数值过大，容易主导分裂，Age的作用被低估。标准化后两特征量级统一，特征评估更均衡，准确率提升。

3、三种算法比较——决策树模型表现

```
from sklearn import tree
clf_tree = tree.DecisionTreeClassifier(criterion='entropy')
clf_tree.fit(X_train, y_train)
```

```
cm = confusion_matrix(y_test, y_pred)
print("决策树混淆矩阵：")
print(cm)
```

决策树混淆矩阵：

```
[[53  5]
 [ 3 19]]
```

```
accuracy_tree = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
print("决策树准确率：", accuracy_tree)
```

决策树准确率： 0.9

决策树

实验结果显示，决策树在测试集上的准确率为90%，80个样本中正确分类72个，错误分类8个。其中5个未购买被误判为购买，3个购买被误判为未购买，误判类型相对均衡。

3、三种算法比较——KNN模型表现

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
cm_knn = confusion_matrix(y_test, y_pred_knn)
acc_knn = accuracy_score(y_test, y_pred_knn)
print("KNN混淆矩阵：")
print(cm_knn)
print("KNN准确率：", acc_knn)
```

KNN混淆矩阵：

```
[[55  3]
 [ 1 21]]
```

KNN准确率： 0.95

KNN

实验结果显示，KNN在测试集上准确率达95%，80个样本中仅错分4个。其中3个未购买被误判为购买，仅1个购买被漏判，召回率高达95.5%，对购买用户的识别能力突出。

3、三种算法比较——逻辑回归模型

```
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression(random_state=0)
logreg.fit(X_train, y_train)
y_pred_log = logreg.predict(X_test)
cm_log = confusion_matrix(y_test, y_pred_log)
acc_log = accuracy_score(y_test, y_pred_log)
print("逻辑回归混淆矩阵：")
print(cm_log)
print("逻辑回归准确率：", acc_log)
```

逻辑回归混淆矩阵：

```
[[57  1]
 [ 5 17]]
```

逻辑回归准确率： 0.925

逻辑回归

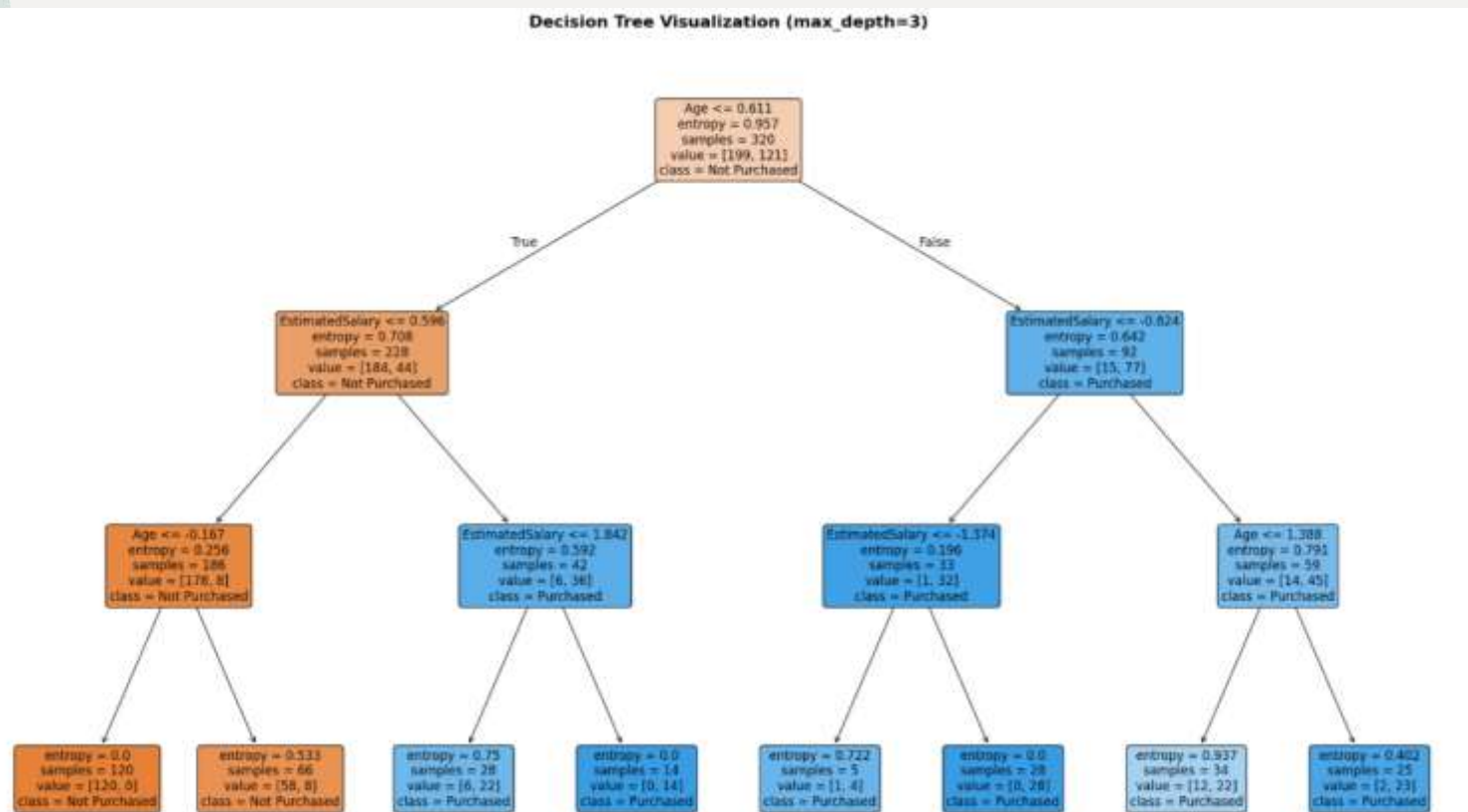
逻辑回归在80个测试样本中正确分类74个，错误6个。其中仅1个未购买被误判，但5个购买被漏判，说明模型偏向于预测“不购买”，对少数类（购买）的识别能力较弱。

3、三种算法比较——三模型准确率对比

模型	准确率
KNN	0.9375
逻辑回归	0.8875
决策树	0.9125

KNN表现最优，该数据集样本在特征空间中分布聚集，KNN基于距离的局部投票机制能有效捕捉相似性
决策树居中，能从数据中学习非线性划分规则，且可解释性强，能直观看到Age和Salary的分裂阈值
逻辑回归表现最弱，因其假设特征是线性可分，而本数据集中Age和Salary与购买行为的关系并非简单线性，导致欠拟合

4、决策树可视化



```
from sklearn.tree import plot_tree
plt.figure(figsize=(20, 12))
plot_tree(clf_viz,
          feature_names=['Age', 'EstimatedSalary'],
          class_names=['Not Purchased', 'Purchased'],
          filled=True,
          rounded=True,
          fontsize=12,
          impurity=True,
          proportion=False)

plt.title('Decision Tree Visualization (max_depth=3)',
          fontsize=16, fontweight='bold')
plt.tight_layout()
plt.savefig('decision_tree.png', dpi=150, bbox_inches='tight')
plt.show()

print("\nDecision Tree Structure:")
print("-" * 50)
print("Node information:")
print("  · Line 1: Split condition (e.g. Age <= 0.325)")
print("  · entropy: Entropy (impurity)")
print("  · samples: Number of samples")
print("  · value: Samples per class [Not Purchased, Purchased]")
print("  · class: Predicted class")
print("-" * 50)
```

Decision Tree Structure:

Node information:

- Line 1: Split condition (e.g. Age <= 0.325)
- entropy: Entropy (impurity)
- samples: Number of samples
- value: Samples per class [Not Purchased, Purchased]
- class: Predicted class

上图展示了最大深度为3的决策树结构。根节点以Age进行首次分裂，说明年龄是预测购买行为最重要的特征。

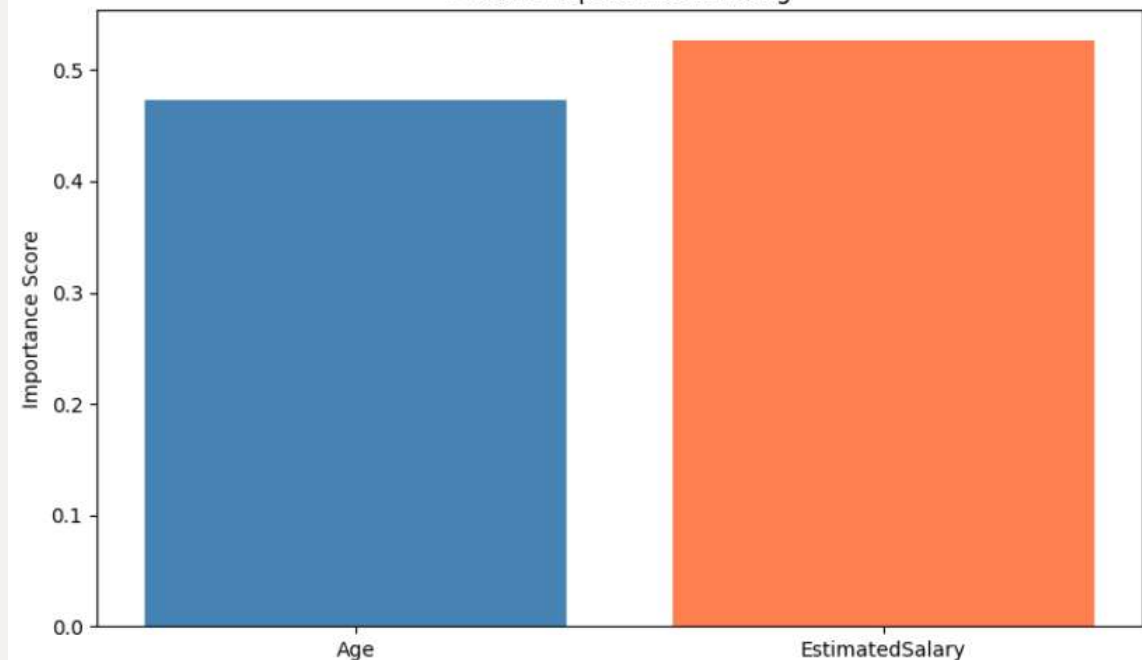
4、决策树可视化与模型评估

```
from sklearn.metrics import classification_report
print("\n分类报告：")
print(classification_report(y_test, y_pred, target_names=['Not Purchased', 'Purchased']))
```

分类报告：

	precision	recall	f1-score	support
Not Purchased	0.95	0.91	0.93	58
Purchased	0.79	0.86	0.83	22
accuracy			0.90	80
macro avg	0.87	0.89	0.88	80
weighted avg	0.90	0.90	0.90	80

Feature Importance Ranking



```
plt.figure(figsize=(8, 5))
plt.bar(['Age', 'EstimatedSalary'], feature_importance, color=['steelblue', 'coral'])
plt.title('Feature Importance Ranking')
plt.ylabel('Importance Score')
plt.tight_layout()
plt.show()
```

决策树总体准确率90%，对未购买用户精确率95%，购买用户召回率86%。特征重要性显示Age远高于Salary，与根节点以Age首次分裂一致。模型可解释性强，但相比随机森林（93.75%）和KNN（95%）仍有差距。



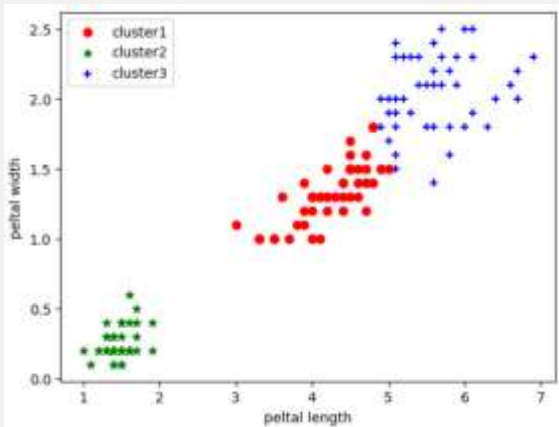
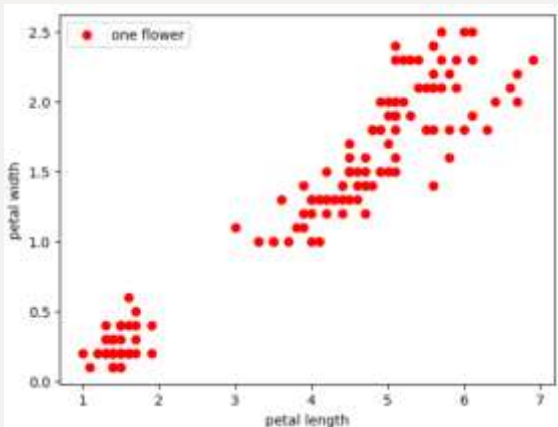
PART 10

K m e a n s - H C

1、用花萼（第0列和第1列）的数据来聚类

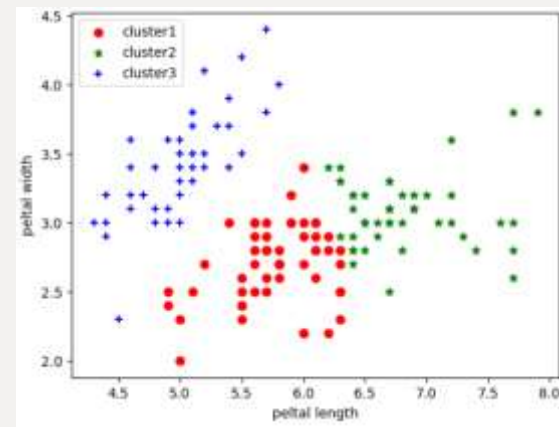
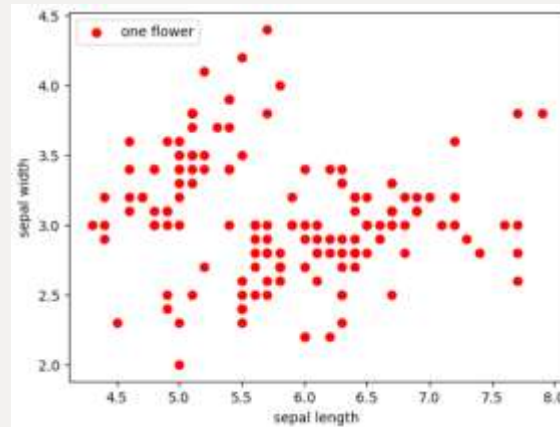
```
X_2 = iris.data[:, 2:4]
```

```
plt.scatter(X_2[:,0],X_2[:,1], c = "red", marker = 'o', label='one flower')  
plt.xlabel('petal length')  
plt.ylabel('petal width')  
plt.legend(loc=2)  
plt.show()
```



```
X_2 = iris.data[:, 0:2]
```

```
plt.scatter(X_2[:,0],X_2[:,1], c = "red", marker = 'o', label='one flower')  
plt.xlabel('sepal length')  
plt.ylabel('sepal width')  
plt.legend(loc=2)  
plt.show()
```

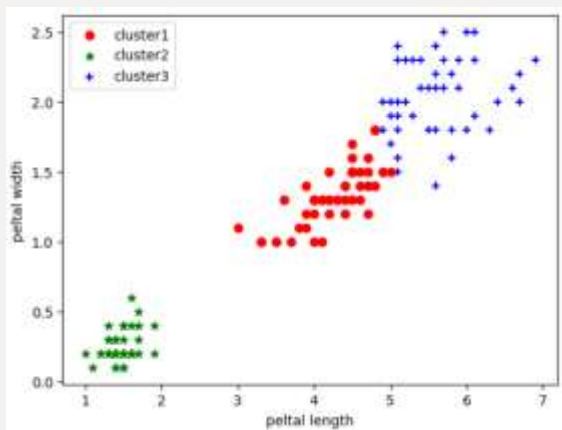
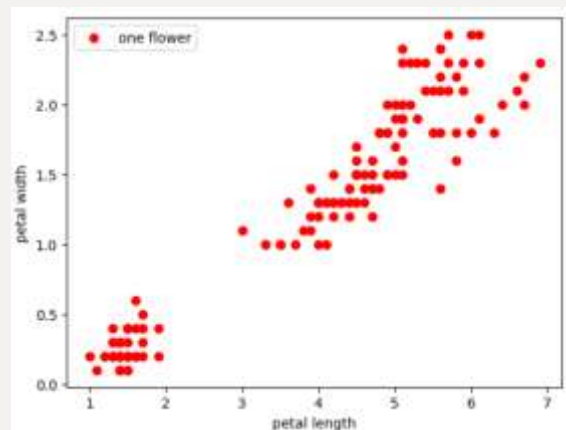


将原代码中的 `X_2 = iris.data[:, 2:4]` 修改为 `X_2 = iris.data[:, 0:2]`，即可用花萼数据（花萼长度和花萼宽度）进行聚类。聚类结果可视化后，与原花瓣数据对比可见：花瓣数据的三个簇分离明显，聚类效果好；花萼数据的三个簇重叠严重，聚类效果较差。这说明鸢尾花数据集中，花瓣特征的分度优于花萼特征。

2、用所有数据来聚类（所有4列）

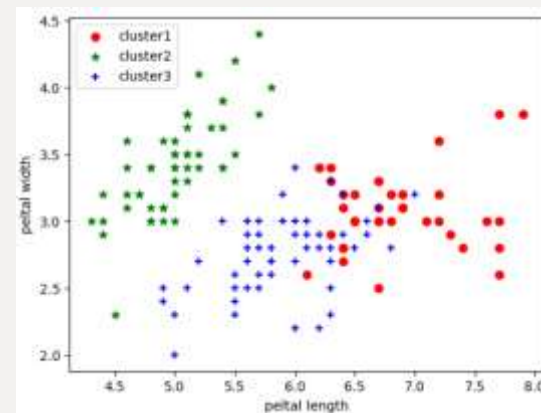
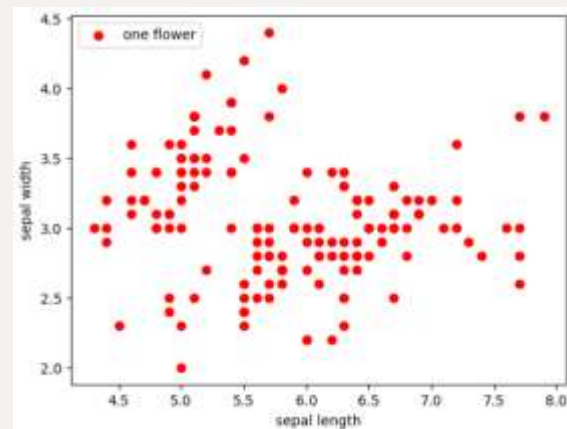
```
X_2 = iris.data[:, 2:4]
```

```
plt.scatter(X_2[:,0],X_2[:,1], c = "red", marker = 'o', label='one flower')
plt.xlabel('petal length')
plt.ylabel('petal width')
plt.legend(loc=2)
plt.show()
```



```
X_2 = iris.data[:, :]
```

```
plt.scatter(X_2[:,0],X_2[:,1], c = "red", marker = 'o', label='one flower')
plt.xlabel('petal length')
plt.ylabel('petal width')
plt.legend(loc=2)
plt.show()
```



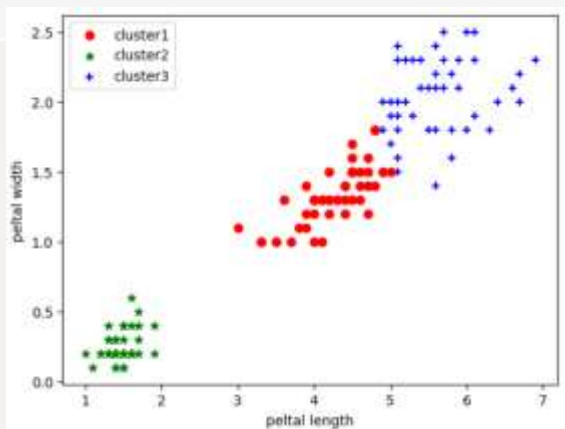
通过将 `iris.data[:, 2:4]` 修改为 `iris.data[:, :]`，即可使用全部4个特征进行聚类。虽然可视化时仍需用2维展示，但聚类过程利用了更完整的信息，通常能获得更准确的聚类结果。用轮廓系数可以定量验证：4列数据的轮廓系数高于2列数据。

3、K=3，把K改成2

K=3

```
estimator = KMeans(n_clusters=3)
```

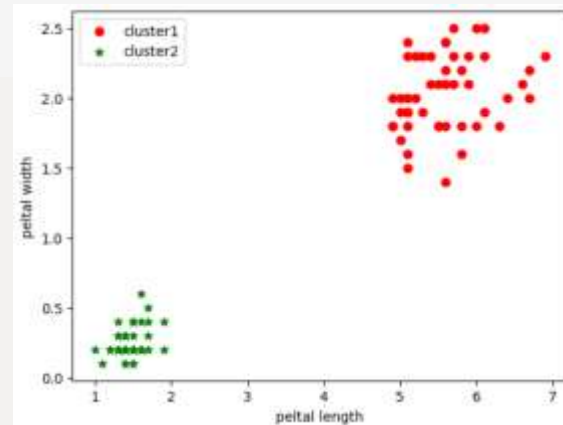
```
plt.scatter(x0[:,0], x0[:,1],c = 'red', marker = 'o', label = 'cluster1')  
plt.scatter(x1[:,0], x1[:,1],c = 'green', marker = '*', label = 'cluster2')  
plt.scatter(x2[:,0], x2[:,1],c = 'blue', marker = '+', label = 'cluster3')  
plt.xlabel('petal length')  
plt.ylabel('petal width')  
plt.legend(loc=2)  
plt.show()
```



K=2

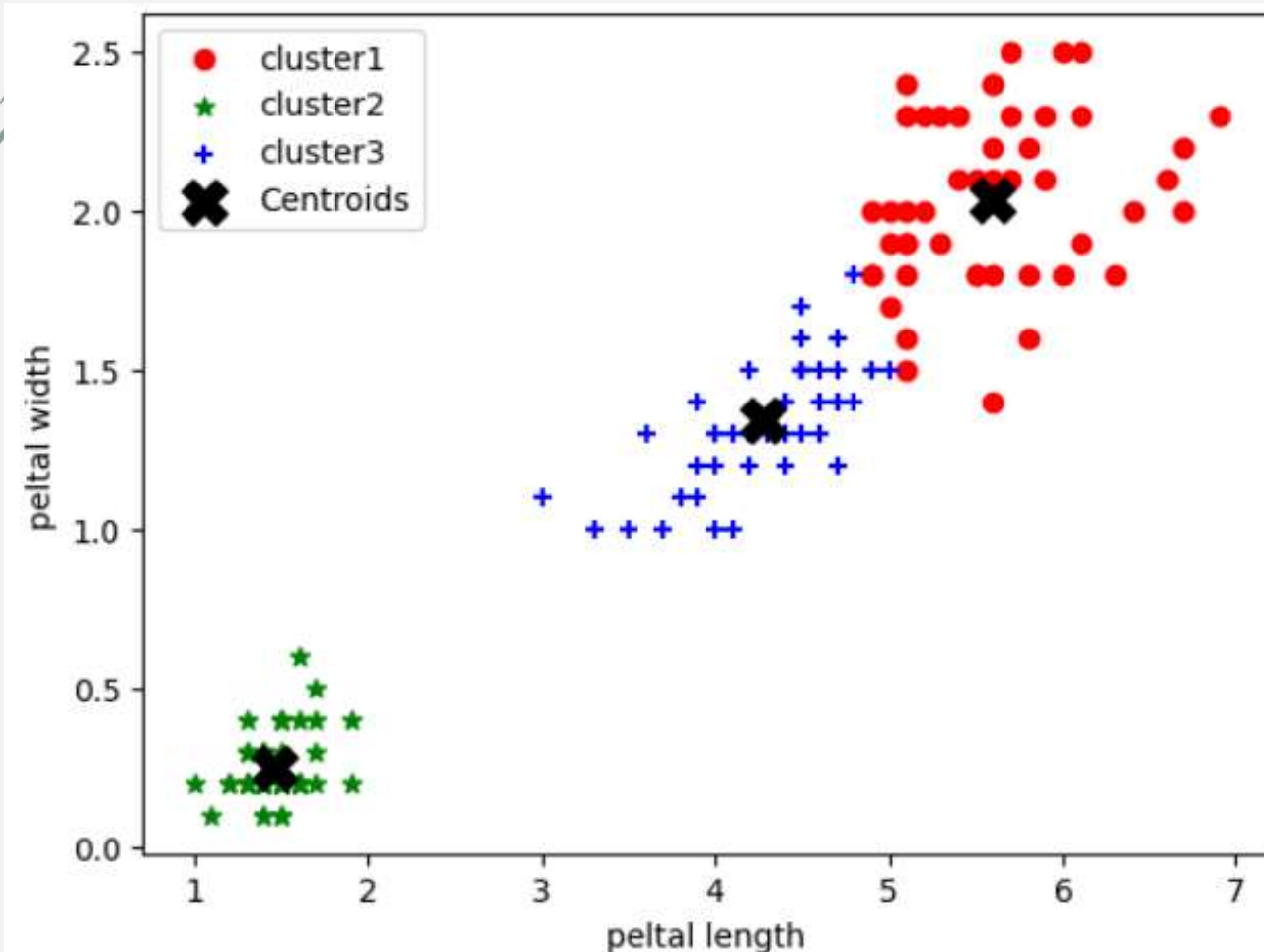
```
estimator = KMeans(n_clusters=2)
```

```
plt.scatter(x0[:,0], x0[:,1],c = 'red', marker = 'o', label = 'cluster1')  
plt.scatter(x1[:,0], x1[:,1],c = 'green', marker = '*', label = 'cluster2')  
#plt.scatter(x2[:,0], x2[:,1],c = 'blue', marker = '+', label = 'cluster3')  
plt.xlabel('petal length')  
plt.ylabel('petal width')  
plt.legend(loc=2)  
plt.show()
```



将 `n_clusters=3` 修改为 `n_clusters=2` 后，聚类结果从3个簇变为2个簇。对比两图可见： $K=3$ 时三个簇分离明显； $K=2$ 时，原本3类中较为接近的两类被合并，另一类保持独立。这体现了 K 值对聚类结果的直接影响—— K 值决定了最终划分的类别数量。

4、聚类中心可视化效果



```
centers = estimator.cluster_centers_  
print("聚类中心坐标：")  
print(centers)
```

```
plt.scatter(x0[:,0], x0[:,1], c = 'red', marker = 'o', label = 'cluster1') # 第一类的花，横  
plt.scatter(x1[:,0], x1[:,1], c = 'green', marker = '*', label = 'cluster2') # 第二类花，绿色  
plt.scatter(x2[:,0], x2[:,1], c = 'blue', marker = '+', label = 'cluster3') # 第三类花，蓝色  
plt.scatter(centers[:, 0], centers[:, 1], c='black', marker='X', s=200, label='Centroids')  
plt.xlabel('petal length')  
plt.ylabel('petal width')  
plt.legend(loc=2)  
plt.show()
```